

Summer Internship Report
On
Artificial Intelligence Internship at CodSoft

Submitted
To
School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of
B.Tech CSE



By
Roll Number of Student: 25SCS1003003551
Name of Student: ESHA KUMARI
Semester: 3rd | Section: 2CSE19
Batch:
2026-27

CANDIDATE'S DECLARATION

I, ESHA KUMARI, do hereby declare that the internship report titled “Artificial Intelligence Internship at CodSoft” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature: Esha Kumari

Student Name: ESHA KUMARI

Roll No.: 25SCS1003003551

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I would like to thank CodSoft for providing me the opportunity to undertake this Artificial Intelligence internship and for the resources, mentorship, and guidance extended throughout the internship period from 10th July 2026 to 10th August 2026. I would also like to thank the faculty of the School of Computer Science and Engineering, IILM University, for their continuous support and encouragement.

I express my gratitude to my parents for their blessings.

Student Name: ESHA KUMARI

Roll No.: 25SCS1003003551

INTERNSHIP COMPLETION CERTIFICATE

C.ID: d62096f



CERTIFICATE

OF COMPLETION
PROUDLY PRESENTED TO

Esha Kumari

has successfully completed 4 weeks of a virtual internship program in

Artificial Intelligence

with wonderful remarks at **CODSOFT** from 10/07/2026 to 10/08/2026.

We were truly amazed by his/her showcased skills and invaluable contributions to the tasks and projects throughout the internship.



A handwritten signature in black ink, appearing to read 'Rajendra'.

Founder



contact@codsoft.in

www.codsoft.in

Date: 11/08/2026

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page Nos.
	Cover Page	—
1.	Candidate's Declaration	i
2.	Acknowledgement	ii
3.	Internship Completion Certificate	—
4.	Project Description	1
	4.1 Introduction	1
	4.2 Organization Profile	2
	4.3 Problem Statement	3
	4.4 Project Objectives	4
	4.5 Scope of the Project	5
	4.6 Technologies and Tools Used	6
	4.7 System Architecture	7
	4.8 Methodology	9
	4.9 Expected Outcomes	15
	4.10 Certificates & Communication Proof	16
5.	Bibliography / References	17

CHAPTER 4

PROJECT DESCRIPTION

4.1 Introduction

This report presents the work carried out during the Artificial Intelligence Virtual Internship undertaken at CodSoft from 10th July 2026 to 10th August 2026. CodSoft's AI internship track is designed to give aspiring developers hands-on, project-based exposure to core Artificial Intelligence concepts — including rule-based natural language processing, adversarial search / game theory, and recommender systems — by requiring interns to independently design, implement and test a minimum of three practical projects using Python.

As part of this internship, three tasks were successfully completed:

- Task 1 – A Rule-Based Chatbot that responds to predefined user queries using pattern/keyword matching.
- Task 2 – A Tic-Tac-Toe AI agent that plays optimally against a human opponent using the Minimax search algorithm, making it unbeatable.
- Task 3 – A Movie Recommendation System that suggests similar movies to a user using content-based filtering with Cosine Similarity.

Each of these tasks strengthened the practical understanding of core AI concepts such as conversational logic, adversarial search algorithms, game theory, and vector-space similarity based recommendation techniques, while also improving proficiency in Python and its associated data-science libraries.

4.2 Organization Profile

CodSoft is an innovative tech education and software development platform designed to empower aspiring developers by transforming theoretical knowledge into practical, hands-on expertise. Through virtual internships, AI-driven career tools, and project-based learning curriculums, CodSoft provides a holistic ecosystem for the growth of student developers — whether they are building their first application or working on more advanced backend and AI-driven services.

CodSoft also offers a range of career-support tools to interns, including a Resume Builder, ATS Score Checker, AI Interview Coach, Career Roadmap Generator, Portfolio Builder, and one-on-one mentorship sessions, along with placement support and networking opportunities, to help interns transition smoothly from learning to industry readiness.

4.3 Problem Statement

Beginners in Artificial Intelligence often struggle to connect theoretical concepts — such as pattern matching, adversarial search, and similarity-based recommendation — with real, working software. There was a need to design and build small but complete AI-driven applications that individually demonstrate a core AI concept end-to-end: a system that can converse with a user through predefined rules, a game-playing agent that can never lose by reasoning over all possible future game states, and a system that can recommend relevant items to a user purely from existing data patterns, without being explicitly told the answer.

4.4 Project Objectives

- To design and implement a rule-based chatbot capable of understanding basic user queries and responding appropriately using conditional/dictionary-based logic.

- To implement an unbeatable Tic-Tac-Toe AI agent using the Minimax algorithm that always plays the mathematically optimal move.
- To build a content-based Movie Recommendation System that recommends similar movies based on genre similarity using vector representation and cosine similarity.
- To gain hands-on experience with Python programming and libraries such as Pandas and Scikit-learn.
- To understand and apply core AI concepts — natural language processing basics, game theory / adversarial search, and content-based filtering — in real, working code.

4.5 Scope of the Project

The scope of this internship was limited to building foundational, console-based AI applications in Python that clearly demonstrate individual AI concepts, rather than production-grade, deployable systems. The chatbot handles a fixed, predefined set of intents and does not perform true natural language understanding. The Tic-Tac-Toe AI is scoped to the standard 3×3 board and two-player (human vs AI) mode. The recommendation system is scoped to content-based filtering over the MovieLens “ml-latest-small” dataset and does not incorporate collaborative/user-rating-based filtering. Together, these three projects give a broad, practical introduction to different sub-fields of Artificial Intelligence within the internship duration.

4.6 Technologies and Tools Used

Table 4.1: Technologies and Tools Used

Category	Tools / Technologies
Programming Language	Python 3
Libraries Used	Pandas, Scikit-learn (CountVectorizer, cosine_similarity), math (inf), datetime
Dataset	MovieLens “ml-latest-small” dataset (movies.csv)

Development Environment	Google Colab (Recommendation System), Python IDE / Terminal (Chatbot & Tic-Tac-Toe)
Version Control / Hosting	Git and GitHub

4.7 System Architecture

Task 1 – Rule-Based Chatbot: The chatbot follows a simple input → processing → output architecture. User input is read, normalized (converted to lowercase and stripped of whitespace), and matched against a Python dictionary of predefined query–response pairs. Special commands (“date”, “time”, “bye”) are handled separately through conditional checks before falling back to the dictionary lookup. If no match is found, a default fallback response is returned.

Task 2 – Tic-Tac-Toe AI: The game architecture consists of a 3×3 board represented as a flat list of 9 cells, a display module, a win/draw checker, a recursive Minimax function that evaluates all possible future game states, and a move-selection function that picks the move with the highest Minimax score for the AI. The human player and AI alternate turns until a win or draw condition is reached.

Task 3 – Movie Recommendation System: The system architecture follows a standard content-based filtering pipeline — the movie dataset is loaded using Pandas, the ‘genres’ column of each movie is vectorized into numerical form using CountVectorizer, a cosine similarity matrix is computed across all movie vectors, and the top-5 most similar movies (excluding the input movie itself) are returned for any given movie title.

4.8 Methodology

4.8.1 Task 1: Rule-Based Chatbot

A dictionary-based approach was used to build the chatbot. On starting, the chatbot greets the user and asks for their name, then displays the list of supported queries. User input is continuously read in a loop; it is converted to lowercase and matched against a dictionary of predefined question–

answer pairs covering greetings, small talk, and general-knowledge questions about Python and AI. Two additional dynamic commands — “date” and “time” — use Python's datetime module to return the current date/time. Typing “bye” ends the conversation.

```
from datetime import datetime
def welcome():
    print("=" * 60)
    print("          Welcome to Rule-Based Chatbot")
    print("=" * 60)

    name = input("Enter your name: ")

    print(f"\nHello, {name}! ")
    print("I'm your AI Chatbot.")
    print("\nYou can ask me:")

    print("""
• hello
• hi
• how are you
• your name
• what is python
• what is ai
• who created python
• what can you do
• date
• time
• bye
""")

    return name

def get_response(user_input):

    responses = {
        "hello": "Hello! How can I help you?",
        "hi": "Hi! Nice to meet you.",
        "how are you": "I'm doing great. Thanks for asking!",
        "your name": "I'm a Rule-Based Chatbot.",
        "what is python": "Python is a popular programming language.",
        "what is ai": "AI stands for Artificial Intelligence.",
        "who created python": "Python was created by Guido van Rossum.",
        "what can you do": "I can answer basic predefined questions and
chat
        with you."
    }

    return responses.get(
        user_input,
        "Sorry, I don't understand that question."
    )

def chatbot():
    name = welcome()

    while True:

        user_input = input(f"\n{name}: ").lower().strip()
```

```

        if user_input == "bye":
            print("\nChatbot: Goodbye! Have a wonderful day.")
            break

        elif user_input == "date":
            today = datetime.now().strftime("%d-%m-%Y")
            print("Chatbot: Today's date is", today)
            continue

        elif user_input == "time":
            current_time = datetime.now().strftime("%H:%M:%S")
            print("Chatbot: Current time is", current_time)
            continue

        response = get_response(user_input)

        print("Chatbot:", response)

chatbot()

```

Figure 4.1: Rule-Based Chatbot – Source Code (chatbot.py)

Sample interaction: the user is greeted by name, asks “hello” and “what is python” and receives the corresponding predefined answers, and the conversation ends when the user types “bye”.

4.8.2 Task 2: Tic-Tac-Toe AI (Minimax Algorithm)

The Tic-Tac-Toe AI was implemented using the classic Minimax algorithm, which explores the complete game tree of remaining moves to determine the optimal move for the AI ('O'), assuming the human opponent ('X') also plays optimally. The recursive minimax() function returns +1 if the AI wins, -1 if the human wins, and 0 for a draw, propagating these scores back up the game tree. On the maximizing player's turn (AI), the function chooses the move that maximizes this score; on the minimizing player's turn (human, simulated), it chooses the move that minimizes it. The ai_move() function tries every empty cell, evaluates the resulting Minimax score, and plays the move with the highest score — guaranteeing the AI never loses.

```

from math import inf

# Create empty board
board = [' ' for _ in range(9)]

```

```

# Display board
def print_board():
    print("\n")
    for i in range(3):
        print(" " + board[i*3] + " | " + board[i*3+1] + " | " +
board[i*3+2])
        if i < 2:
            print("---|---|---")
    print("\n")

# Check winner
def check_winner(player):
    win_conditions = [
        [0,1,2], [3,4,5], [6,7,8],
        [0,3,6], [1,4,7], [2,5,8],
        [0,4,8], [2,4,6]
    ]

    for condition in win_conditions:
        if all(board[pos] == player for pos in condition):
            return True
    return False

# Check draw
def is_draw():
    return ' ' not in board

# Minimax Algorithm
def minimax(is_maximizing):

    if check_winner('O'):
        return 1

    if check_winner('X'):
        return -1

    if is_draw():
        return 0

    if is_maximizing:
        best_score = -inf

        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                score = minimax(False)
                board[i] = ' '
                best_score = max(score, best_score)

        return best_score

    else:
        best_score = inf

        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax(True)
                board[i] = ' '
                best_score = min(score, best_score)

        return best_score

# AI Move
def ai_move():
    best_score = -inf

```

```

best_move = -1

for i in range(9):
    if board[i] == ' ':
        board[i] = 'O'
        score = minimax(False)
        board[i] = ' '

        if score > best_score:
            best_score = score
            best_move = i

board[best_move] = 'O'

# Human Move
def player_move():
    while True:
        try:
            move = int(input("Enter your move (1-9): ")) - 1

            if move < 0 or move > 8:
                print("Please enter a number between 1 and 9.")
                continue

            if board[move] != ' ':
                print("Position already occupied.")
                continue

            board[move] = 'X'
            break

        except ValueError:
            print("Invalid input. Enter a number.")

# Main Game
def play_game():

    print("=" * 40)
    print("      TIC-TAC-TOE AI")
    print("=" * 40)
    print("You are X")
    print("AI is O")
    print("\nBoard Positions:")
    print("1 | 2 | 3")
    print("4 | 5 | 6")
    print("7 | 8 | 9")

    while True:

        print_board()

        player_move()

        if check_winner('X'):
            print_board()
            print(" Congratulations! You Win!")
            break

        if is_draw():
            print_board()
            print(" It's a Draw!")
            break

        print("AI is thinking...\n")
        ai_move()

```

```

        if check_winner('O'):
            print_board()
            print(" AI Wins!")
            break

        if is_draw():
            print_board()
            print(" It's a Draw!")
            break

# Start Game
play_game()

```

Figure 4.2: Tic-Tac-Toe AI – Source Code (tic_tac_toe.py)

The board is displayed as a 3×3 grid after every move; the human enters a position from 1–9, and win/draw conditions are checked after each turn for both players.

4.8.3 Task 3: Movie Recommendation System (Content-Based Filtering)

The recommendation system was built using the MovieLens “ml-latest-small” dataset. The movies.csv file, containing movie titles and their associated genres, is loaded into a Pandas DataFrame. Missing genre values are filled with empty strings to avoid errors. Scikit-learn's CountVectorizer is then used to convert each movie's genre string into a numerical vector (a bag-of-words representation over genre tokens). Cosine similarity is computed between every pair of movie vectors, producing a similarity matrix where a higher score indicates greater genre overlap. Given an input movie title, the recommend() function locates the movie's index, sorts all other movies by their similarity score in descending order, and returns the top 5 most similar titles (excluding the movie itself).

```

import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

movies = pd.read_csv('ml-latest-small/movies.csv')
movies['genres'] = movies['genres'].fillna('')

cv = CountVectorizer()
matrix = cv.fit_transform(movies['genres'])

similarity = cosine_similarity(matrix)

def recommend(movie_name):

```

```

try:
    index = movies[movies['title'] == movie_name].index[0]
    distances = list(enumerate(similarity[index]))
    movie_list = sorted(distances, reverse=True, key=lambda x:
x[1])[1:6]

    print("Recommended Movies:\n")
    for movie in movie_list:
        print(movies.iloc[movie[0]].title)
except:
    print("Movie not found!")

recommend("Toy Story (1995)")

movie_name = input("Enter Movie Name: ")
recommend(movie_name)

```

Figure 4.3: Movie Recommendation System – Source Code

For example, given the input “Toy Story (1995)”, the system recommends genre-similar animated/family movies such as “Antz (1998)”, “Toy Story 2 (1999)”, and “Monsters, Inc. (2001)”, demonstrating that the content-based approach successfully groups movies with overlapping genres.

4.9 Expected Outcomes

- A fully working rule-based chatbot capable of holding a basic, structured conversation with a user and responding correctly to all predefined queries.
- An unbeatable Tic-Tac-Toe AI that either wins or draws every game against a human opponent, correctly demonstrating the Minimax algorithm and game-tree search.
- A functioning content-based Movie Recommendation System that returns relevant, genre-similar movie recommendations for any valid movie title present in the dataset.
- Strengthened practical skills in Python programming, use of Pandas and Scikit-learn, recursive algorithm design, and applied understanding of core AI paradigms (rule-based systems, adversarial search, and content-based filtering).

- All three projects version-controlled and publicly hosted on GitHub as required by the internship submission guidelines.

4.10 Certificates of Completion and Other Communication Mail Proof

The Internship Completion Certificate issued by CodSoft is attached in the preliminary pages of this report. Screenshots of task-submission confirmation communication from CodSoft, and the LinkedIn post(s) showcasing the completed projects, should be attached below as image.

Esha Kumari · You
UG CSE with AI|| C Programming || C ++ with OOP|| Basic ...
1mo · Edited ·

From "Why isn't this code working?" to "Project Completed." 😊

Successfully completed my Tic-Tac-Toe AI project for Task 2 of the CODSOFT AI Internship.

Each project reminds me that growth doesn't happen in a single breakthrough—it happens through consistent effort, mistakes, and improvements.

Task-3 coming soon. 🚀 **CodSoft CodSoft Internships**

#CODSOFT #AIInternship #Python #ArtificialIntelligence
#DeveloperJourney #TechLearning #WomenInTech



The project I have developed is a Tic
tac toe AI using the

Esha Kumari · You
UG CSE with AI|| C Programming || C ++ with OOP|| Basic ...
1mo · Edited ·

🚀 Excited to share my first task completed during the CODSOFT Artificial Intelligence Internship!

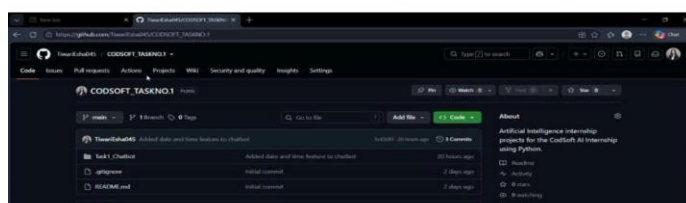
I developed an AI Chatbot using Python that interacts with users through predefined responses and includes features like displaying the current date and time. This project helped me strengthen my Python programming skills and gain hands-on experience in building conversational AI applications. In this video, I've demonstrated the project, explained the implementation, and showcased the source code.

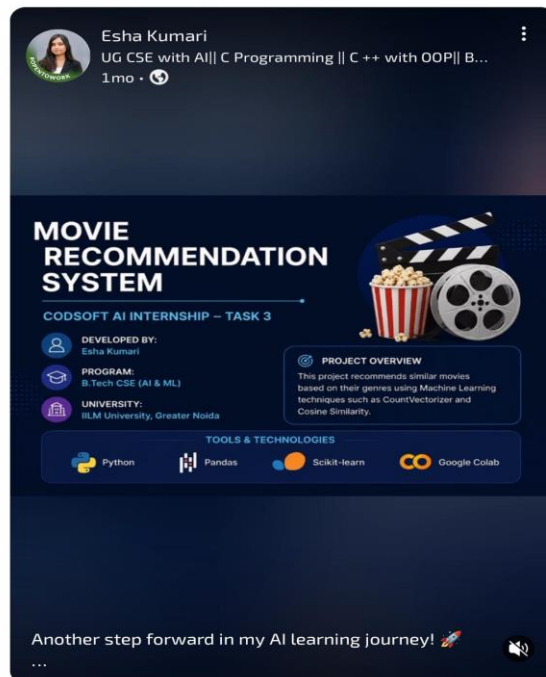
🔗 GitHub Repository: <https://lnkd.in/dh-Chr5i>

I'd love to hear your feedback and suggestions!

Thank you, **CodSoft**, for providing this learning opportunity.

#CODSOFT #ArtificialIntelligence #Python #AIChatbot
#MachineLearning #Programming #SoftwareDevelopment
#GitHub #OpenSource #Tech #Coding #Developer
#ComputerScience #BTech #StudentDeveloper
#PythonDeveloper #AI #Innovation #Learning #Internship
#TechCommunity #CodeNewbie #WomenInTech #FutureTech
#ProjectShowcase CodSoft Internships CODSOFT Group





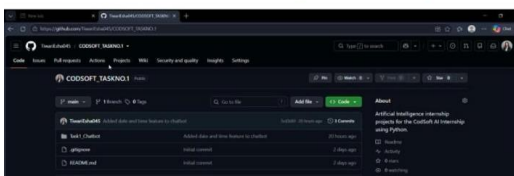
I developed an AI Chatbot using Python that interacts with users through predefined responses and includes features like displaying the current date and time. This project helped me strengthen my Python programming skills and gain hands-on experience in building conversational AI applications. In this video, I've demonstrated the project, explained the implementation, and showcased the source code.

GitHub Repository: <https://lnkd.in/dh-Chr5i>

I'd love to hear your feedback and suggestions!

Thank you, **CodSoft**, for providing this learning opportunity.

#CODSOFT #ArtificialIntelligence #Python #AIChatbot
#MachineLearning #Programming #SoftwareDevelopment
#GitHub #OpenSource #Tech #Coding #Developer
#ComputerScience #BTech #StudentDeveloper
#PythonDeveloper #AI #Innovation #Learning #Internship
#TechCommunity #CodeNewbie #WomenInTech #FutureTech
#ProjectShowcase CodSoft Internships COSOFT Group



CHAPTER 5

BIBLIOGRAPHY / REFERENCES

1. CodSoft Internship Program – Official Website. Available at: <https://www.codsoft.in>
2. Python Software Foundation. Python 3 Documentation. Available at: <https://docs.python.org/3/>
3. Pandas Development Team. Pandas Documentation. Available at: <https://pandas.pydata.org/docs/>
4. Scikit-learn Developers. Scikit-learn: Machine Learning in Python – CountVectorizer and Cosine Similarity Documentation. Available at: <https://scikit-learn.org/stable/>
5. GroupLens Research. MovieLens “ml-latest-small” Dataset. Available at: <https://grouplens.org/datasets/movielens/>
6. Russell, S., & Norvig, P. Artificial Intelligence: A Modern Approach (Chapter on Adversarial Search / Minimax Algorithm).
7. Author's Project Repositories – GitHub:
https://github.com/TiwariEsha045/CODSOFT_TASKNO.1 ,
https://github.com/TiwariEsha045/CODSOFT_TASKNO2 ,
https://github.com/TiwariEsha045/CODSOFT_Task3_Recommendation_System